TECHNICAL PROJECT SUMMARY

ExpenseAI

Complete documentation of technologies, features, backend, database, login, AI, and macOS app packaging.

Field	Details
Project name	ExpenseAI (expense-ai)
Type	Smart expense tracker with AI features
Prepared on	May 18, 2026
Project folder	/Users/aswinsanthosh/Downloads/src 3
Desktop app	ExpenseAI.app (macOS bundle)

One-line summary

ExpenseAI is a full-stack local app: React + TypeScript frontend, Python HTTP backend, SQLite database, secure login with sessions, AI category suggestions, receipt OCR scanning, analytics charts, and a macOS .app that launches everything on localhost.

1. Executive Summary

- ExpenseAI tracks personal expenses with AI-assisted categorization, spending insights, and receipt scanning.
- Users sign up or log in with email and password. All expenses are stored per user in a SQLite database.
- The frontend is a React single-page app built with Vite and styled with Tailwind CSS.
- The backend is a custom Python server (no Flask/Django) that exposes a REST-style JSON API.
- Receipt photos are read in the browser using Tesseract.js OCR, then parsed locally.
- A macOS desktop app (ExpenseAI.app) bundles the built site plus backend and opens it in the browser.

2. Technology Stack

Area	Technology	Where / Files	Purpose
Frontend	React 18.3 + TypeScript 5.6	src/App.tsx, src/main.tsx	Interactive dashboard UI
Build tool	Vite 6 + @vitejs/plugin-react	vite.config.ts	Dev server and production build (dist/)
Styling	Tailwind CSS 3 + PostCSS	tailwind.config.js, src/index.css	Responsive layout, dark/light theme
UI components	Custom shadcn-style (Button, Card, Input, Label)	src/components/ui/*	Reusable form and layout primitives
Icons	lucide-react	Throughout components	Sparkles, Camera, Moon, Sun, etc.
Backend	Python 3 stdlib (http.server)	backend/server.py	REST API + optional static file serving
Database	SQLite 3	backend/data/expenseai.sqlite3	Users, sessions, expenses
OCR	Tesseract.js 6	ReceiptScanPanel.tsx	Read text from receipt images in browser
AI logic	Local rule-based engine	src/utils/aiEngine.ts	Category suggestions and spending insights
Desktop app	macOS .app + LaunchAgent	ExpenseAI.app/Contents/*	Double-click to launch on Mac

3. Backend (Python API Server)

The backend runs at http://127.0.0.1:8787 during development. It uses Python's built-in ThreadingHTTPServer — no external web framework.

How to start

- npm run backend — starts only the Python API server
- npm run dev:full — starts backend + Vite frontend together (via backend/dev.py)
- npm run serve:full — serves built frontend from dist/ through the backend

API Endpoints

Method	Route	Auth required	What it does
GET	/api/health	No	Health check; returns database path
POST	/api/auth/register	No	Create account; returns user + session token
POST	/api/auth/login	No	Sign in; returns user + session token
GET	/api/auth/me	Yes (Bearer)	Return current logged-in user
POST	/api/auth/logout	Yes	Invalidate session token
GET	/api/expenses	Yes	List all expenses for the user
POST	/api/expenses	Yes	Create a new expense
PUT	/api/expenses/{id}	Yes	Update an expense
DELETE	/api/expenses/{id}	Yes	Delete an expense

Security features

- Passwords hashed with PBKDF2-HMAC-SHA256 (120,000 iterations) + random salt
- Session tokens are random URL-safe strings; only SHA-256 hashes stored in DB
- Sessions expire after 30 days; expired sessions cleaned on each request
- Bearer token sent in Authorization header from frontend
- CORS headers enabled for dev proxy
- Email validation and minimum 6-character password enforced server-side

4. Database (SQLite)

Database file location

Development: backend/data/expenseai.sqlite3. Desktop app: ~/Library/Application Support/ExpenseAI/expenseai.sqlite3

Table	Columns / purpose
users	id, email (unique), password_hash, password_salt, created_at
sessions	token_hash (PK), user_id, created_at, expires_at — login sessions
expenses	id, user_id, description, amount, category, date, notes, ai_suggested, created_at, updated_at

Foreign keys link sessions and expenses to users. Expenses are scoped per user — each user only sees their own data.

5. Frontend Features

Feature	Component / File	Description
Main dashboard	App.tsx	Login gate, header, stats, expense form, list, insights, charts
Add expense	ExpenseForm.tsx	Description, amount, date, category, notes; AI suggestion on type
Expense list	ExpenseList.tsx	View, inline edit, delete expenses
Stats overview	StatsOverview.tsx	Total spent, this month, AI-categorized count with detail modals
Spending chart	SpendingChart.tsx	Category totals as progress bars
Pie chart	CategoryPieChart.tsx	SVG donut chart with time filters and search
AI insights	InsightsPanel.tsx	Warnings, tips, spending projections from aiEngine
Receipt scan	ReceiptScanPanel.tsx	Upload photo, camera, webcam; Tesseract OCR
Receipt parser	receiptParser.ts	Extract merchant, total, line items from OCR text
Dark / light theme	App.tsx + index.css	Toggle stored in localStorage (theme key)
API client	lib/api.ts	fetch wrapper with Bearer token for all backend calls

6. Login & Authentication

Login page (Login.tsx)

- Single card UI for both Sign In and Sign Up (toggle between modes)
- Fields: email and password with validation (email must contain @, password min 6 chars)
- Uses shared Input, Button, Label, Card components with dark mode support
- Emerald gradient background; loading states while API request runs
- On success, App.tsx loads user expenses from backend and shows dashboard

Session flow

1. User submits login or register form
2. Frontend calls POST /api/auth/login or /api/auth/register
3. Backend returns { user, token }; frontend stores token in localStorage (expenseai_api_token)
4. All later API calls include Authorization: Bearer <token>
5. On app load, if token exists, GET /api/auth/me and GET /api/expenses restore session
6. Logout calls POST /api/auth/logout and clears localStorage token

Vite dev proxy

During development, Vite proxies /api requests to http://127.0.0.1:8787 so the frontend and backend work together without CORS issues.

7. AI Features

> **Important: no cloud LLM**
>
> All AI behavior runs locally in the browser. No OpenAI, Gemini, or external API calls. Category suggestions use keyword matching; insights use math on expense data; receipt reading uses Tesseract OCR.

AI category suggestions (aiEngine.ts)

- suggestCategory(description) tokenizes text and scores against keyword lists per category
- Categories: Food & Dining, Transportation, Shopping, Entertainment, Bills & Utilities, Healthcare, Education, Travel, Other
- Recognizes merchants: Zomato, Swiggy, Amazon, Uber, Netflix, etc.
- Returns: category, confidence (0–0.96), reason, matched signals, practical action tip
- Shown in ExpenseForm as user types description (300ms debounce)

AI spending insights

- generateInsights: category concentration warnings, weekly daily average, monthly projection
- calculateCategoryTotals: totals and percentages for charts
- Smart tips (e.g. meal prepping if food spend is high)

Receipt OCR & parsing

Step	Technology	What happens
1. Image input	File upload, device camera, or live webcam	User provides receipt photo
2. Preprocessing	HTML Canvas	Grayscale, contrast boost, upscale for better OCR
3. OCR	Tesseract.js (English worker)	Extracts raw text from image
4. Parsing	receiptParser.ts	Finds merchant, total amount, line items, confidence
5. Auto-fill	ExpenseForm	Fills description, amount, notes; triggers AI category suggestion
6. Verify	UI warnings	User warned to verify OCR before saving

8. macOS Desktop App (ExpenseAI.app)

How we made the Mac app

We did NOT use Electron. The Vite production build was copied into a standard macOS .app bundle. A zsh launcher script starts a local Python backend (with SQLite) that also serves the static React site, then opens the default browser to the app URL.

Part	Path	Purpose
App bundle	ExpenseAI.app	Double-clickable Mac application
Info.plist	Contents/Info.plist	Bundle ID com.local.expenseai, version 1.0.0
Launcher	Contents/MacOS/ExpenseAI	zsh script: start server, open browser
Built frontend	Contents/Resources/site/	index.html + Vite JS/CSS assets
Backend copy	Contents/Resources/backend/server.py	Python API server for desktop
Node fallback	Contents/Resources/server.mjs	Static server if Python unavailable
LaunchAgent	~/Library/LaunchAgents/com.local.expenseai.server.plist	Auto-starts server on login
App URL	http://127.0.0.1:8000/index.html	Opens in default browser
Desktop DB	~/Library/Application Support/ExpenseAI/expenseai.sqlite3	Per-user data on Mac

Desktop launch flow

1. User double-clicks ExpenseAI.app
2. Launcher checks http://127.0.0.1:8000/api/health
3. If not running, writes LaunchAgent plist and starts Python backend with --static-dir and --db
4. Waits up to ~6 seconds for server ready
5. Opens http://127.0.0.1:8000/index.html in default browser

9. How to Run the Project

Command	What it does
npm install	Install frontend dependencies
npm run dev:full	Start backend (8787) + Vite dev server (5173) together
npm run dev	Frontend only (needs backend running separately)
npm run backend	Python API only on port 8787
npm run build	TypeScript check + production build to dist/
npm run serve:full	Serve dist/ through backend on port 8787

10. Project File Structure

Path	Role
src/App.tsx	Root component: auth, theme, layout, state
src/lib/api.ts	Backend API client (auth + expenses)
src/components/auth/Login.tsx	Login / sign-up page
src/utils/aiEngine.ts	Category AI + insights engine
src/utils/receiptParser.ts	Receipt text parser
backend/server.py	Python HTTP API + SQLite
backend/dev.py	Runs backend + Vite together
backend/data/	SQLite database folder
ExpenseAI.app/	macOS desktop application bundle
dist/	Production build output
reports/	This PDF and build scripts

11. npm Dependencies

Package	Version	Used for
react / react-dom	18.3.1	UI framework
vite	6.0.3	Build and dev server
typescript	5.6.3	Type safety
tailwindcss	3.4.15	Utility CSS
tesseract.js	6.0.0	Receipt OCR
lucide-react	0.468.0	Icons
clsx + tailwind-merge	latest	Conditional CSS classes